

Importing Data in R: Exploring Various Packages and Their Advantages: Data Analysis for Decision-Making (SS 2026)

Ali Soleimani (Matriculation Nr.: 401166548)
Fresenius University of Applied Science

Abstract

This document describes the importance of importing data in R, introducing the packages and their advantages.

1 Why Do We Import Data?

In the modern and dynamic report creation procedure, importing data is important to have quick and reliable access to the recent available data. R program helps us to import these data easily and prepare the routine reports quickly. Also data import is the key for starting Data science, which is then followed by tidying data, transforming it, visualizing, and modeling it Wickham and Golemund (2016).

- **Accessing the “Raw” Reality:** Decisions are based on data stored in diverse environments—from simple text files and legacy spreadsheets to massive relational databases and specialized scientific instruments R Core Team (2015), Gruson et al. (2019).
- **The Reproducibility Imperative:** A core advantage of using R over “point-and-click” software is reproducibility. In fields like public health or climate change, where decisions have high stakes, open-source scripts allow every step of the data import and analysis to be audited and checked by others Bivand et al. (2008), “Chapter 2 - Data Import and Export in R and Python” (2019).
- **Efficiency and Automation:** Manual data entry or GUI-based exports are prone to human error and are not scalable. R enables the creation of scripts that can be replayed and modified for new datasets with minimal effort, providing a high return on the initial “steep learning curve” investment Bivand et al. (2008), Huber (2025).
- **Handling Complexity:** R allows decision-makers to overcome the limitations of traditional software like Excel, particularly when dealing with non-rectangular data, complex encodings, or datasets that exceed standard spreadsheet row limits Huber (2025), R Core Team (2015).

2 Different Data Types

Before choosing a package, a data scientist must understand the structure of the source material. Data generally falls into several categories:

1. **Rectangular/Tabular Data:** This is the most common type, where data is organized into rows (observations) and columns (variables). Examples include Comma Separated Values (.csv) and Tab Delimited (.txt or .tsv) files R Core Team (2015), Wickham and Grolemund (2016).
2. **Proprietary Spreadsheets:** Formats like Microsoft Excel (.xls, .xlsx) are ubiquitous in business but complex because they contain multiple sheets, formulas, and macros rather than just raw data Luraschi (2023), R Core Team (2015).
3. **Statistical Software Files:** Data often originates from legacy systems like **SAS** (.sas7bdat), **SPSS** (.sav), or **Stata** (.dta). These files often contain important metadata, such as value labels, which must be preserved during import Chapman University (2026), Luraschi (2023).
4. **Relational Databases:** For very large-scale data that does not fit into a computer's RAM, data is stored in RDBMS like **MySQL**, **PostgreSQL**, or **SQLite**. Accessing these requires specialized connection protocols R Core Team (2015).
5. **Spatial Data:** Includes coordinates, bounding boxes, and reference systems. These are categorized into **vector** models (points, lines, polygons) and **raster** models (regular grids like satellite imagery) Bivand et al. (2008).
6. **Scientific/Specialized Data:** Data from instruments like spectrophotometers often comes in proprietary formats that require specialized parsers to extract both measurements and critical metadata Gruson et al. (2019).

3 Different Packages For Each Data Type

R's ecosystem offers a specialized collection of tools for data acquisition, ranging from built-in **Base R** functions to the modern, high-performance **Tidyverse** suite.

Table 1 summarizes the primary packages used for various data formats in professional data science workflows.

Using this specialized ecosystem ensures that data scientists can maintain **reproducibility**—a core advantage of R—by documenting every step of the import process in a script rather than relying on manual “point-and-click” methods Bivand et al. (2008), Huber (2025), “Chapter 2 - Data Import and Export in r and Python” (2019).

3.1 Graphical and Programmatic Implementation by Data Type

This section details the practical application of the tools identified in the previous page. For each category, we contrast the **Graphical User Interface (GUI)** approach—often best for initial exploration—with the **Programmatic R Code** required for reproducible scripts.

3.1.1 Standard Tabular Data (.csv, .tsv)

- **Graphical:** Use the RStudio “**Import Dataset**” wizard and select “**From Text (readr)**”. This allows for a live preview where you can visually change delimiters, skip rows, and rename columns while the GUI generates the code automatically Luraschi (2023).
- **Code Wickham and Grolemund (2016) :**

```
Rows: 2 Columns: 2
```

```
-- Column specification -----
Delimiter: ", "
```

Table 1
Overview of data import packages

Data Category	File Extensions	Key Package	Primary Functions	Advantages / Notes
Standard Tabular	.csv, .tsv, .txt	readr	read_csv(), read_tsv()	Fast import; creates tibbles and supports large-file progress bars. Wickham et al. (2019), Chapman University (2026).
Proprietary Excel	.xls, .xlsx	readxl	read_excel(), excel_sheets()	Reads Excel files without Java; supports sheet selection. Luraschi (2023), Chapman University (2026).
Statistical Files	.sav, .sas7bdat, .dta	haven	read_sas(), read_sav(), read_dta()	Preserves metadata and labels. Chapman University (2026), Luraschi (2023).
Relational DBs	.db, .sqlite, .mdb	DBI	dbConnect(), dbGetQuery()	SQL interface for large datasets. R Core Team (2015).
Spatial / GIS	.shp, .tif, .kml	sf	st_read()	Handles spatial data with CRS support. Bivand et al. (2008).
Base Text	.txt, .dat, .fwf	utils	read.table(), read.csv(), read.fwf()	Base R functions for legacy formats. R Core Team (2015).
Scientific Data	.jdx, .abs, .cnv, .nc	lightr	lr_get_spec()	Spectral data import. Gruson et al. (2019).
Interactive	Various	speedR	speedR()	GUI-based filtering and code generation. Visne et al. (2012).
Cloud Sheets	URL	googlesheets4	read_sheet()	Imports Google Sheets. Chapman University (2026).
Hierarchical Data	.json	jsonlite	fromJSON()	JSON / API data handling. R Core Team (2015).

dbl (2): x, y

- i Use ``spec()`` to retrieve the full column specification for this data.
- i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

large CSV:

3.1.2 *Proprietary Excel Files (.xls, .xlsx)*

- **Graphical:** Select “**From Excel**” in the RStudio wizard. This is crucial for navigating multi-sheet workbooks, as you can select specific sheets and define a cell range (e.g., “A1:E20”) graphically Luraschi (2023).
- **Code** Chapman University (2026):

3.1.3 *Statistical Software Files (.sav, .sas7bdat, .dta)*

- **Graphical:** Select “**From SPSS/SAS/Stata**” in RStudio. This wizard is particularly helpful as it shows how proprietary labels and missing value codes will be translated into R factors Luraschi (2023).
- **Code R Core Team** (2015):

3.1.4 *Relational Databases (.db, .sqlite, .mdb)*

- **Graphical:** Access through the RStudio “**Connections**” pane. Once a DBI connection is established, you can browse table schemas and preview data before importing Luraschi (2023).
- **Code R Core Team** (2015):

3.1.5 *Spatial / GIS Data (.shp, .tif, .kml)*

- **Graphical:** Usually handled programmatically, though GIS software like GRASS can be linked to R to browse spatial layers.
- **Code Bivand et al.** (2008) :

Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE

3.1.6 *Base Text and Fixed-Width Files (.txt, .dat, .fwf)*

- **Graphical:** Use the “**From Text (base)**” option in the RStudio wizard for legacy text formats that require manual field-width definitions Luraschi (2023).
- **Code R Core Team** (2015):

3.1.7 *Scientific Spectral Data (.jdx, .abs, .cnv, .nc)*

- **Graphical:** Utilize specialized package GUIs or high-level functions that automate folder-wide imports Gruson et al. (2019).
- **Code Gruson et al.** (2019) :

Warning: No files found. Try a different value for argument "ext".

3.1.8 *Interactive Import (speedR)*

- **Graphical:** The **speedR** package provides a standalone GUI for filtering and subsetting large datasets before they are loaded into the R environment Visne et al. (2012).
- **Code:** Visne et al. (2012)

3.1.9 *Cloud Sheets (Google Sheets/ URLs)*

- **Code:** Chapman University (2026)

```
v Reading from "SS".

v Range 'sample_sheet'.
```

3.1.10 *Hierarchical Data (.json)*

- **Graphical:** While RStudio does not have a dedicated “JSON Wizard” in the same way it does for Excel or CSV, users can often preview JSON structures through the **File Pane** or by using specialized viewing packages.
- **Code R Core Team (2015) :**

3.2 Importing Data From Multiple Files

Decision-makers often face the need to import hundreds of files simultaneously. R provides several automation strategies Wickham and Grolemund (2016), Gruson et al. (2019).

- **Automated Recursive Import:** Specialized tools like **lightr** use functions such as `lr_get_spec()` to search subfolders automatically for specific extensions and process them using **parallelized loops** to save time Gruson et al. (2019).
- **The Tidyverse Approach (Iteration):** The most flexible method combines `list.files()` with the **purrr** package. By using `map()` or `map_df()`, an analyst can apply a read function to every path in a list, automatically stacking them into a single unified tibble Wickham et al. (2019).
- **Complex Multi-File Formats:** Some formats are naturally batch-oriented. For example, a single “Shapefile” is a collection of files (`.shp`, `.dbf`, etc.). The `readOGR()` function in **rgdal** uses a **Data Source Name (DSN)**—often a folder path—to treat these multiple files as a single spatial object Bivand et al. (2008), R Core Team (2015).
- **Custom Functions for Productivity:** **speedR** can graphically define an import process and then **generate a new R function** containing those specific constraints. This “ready-to-use” code can be applied to any number of similar files to ensure a consistent pipeline Visne et al. (2012).

3.3 The Comparison Between Packages

Selecting the appropriate import method is a strategic decision that impacts the speed, reliability, and reproducibility of the analytical pipeline Wickham and Grolemund (2016). Table 1 provides a side-by-side comparison of the key packages discussed in this handnote to help choosing the right tool based on the specific data requirements.

Key Decision-Making Guidelines for the Presentation:

Table 1*The Comparison Between Packages*

Data Category	File Formats	Recommended Package	Key Function	Graphical Method (GUI)	Main Advantage / Limitation
Tabular Data	.csv, .tsv, .txt	readr	read_csv()	Import Dataset -> From Text (readr)	Very fast import; creates tibbles / Limited by RAM. Wickham et al. (2019), Luraschi (2023)
Excel Sheets	.xls, .xlsx	readxl	read_excel()	Import Dataset -> From Excel	No Java required; supports multiple sheets / Limited scalability. Luraschi (2023), R Core Team (2015).
Statistical Files	.sav, .dta, .sas7bdat	haven	read_sav(), read_dta()	Import Dataset -> From SPSS/Stata	Preserves labels and metadata / Limited newest-version support. Chapman University (2026), Luraschi (2023).
Databases (SQL)	.db, .sqlite, MySQL	DBI	dbConnect(), dbGetQuery()	RStudio "Connections" Pane	Handles large datasets / Requires SQL knowledge. R Core Team (2015), Chapman University (2026).
Spatial / GIS	.shp, .tif, .kml	rgdal	readOGR(), readGDAL()	External links (e.g., GRASS)	Supports CRS metadata / Complex for beginners. Bivand et al. (2008), R Core Team (2015).
Hierarchical (Web)	.json	jsonlite	fromJSON()	N/A (Script-based)	Standard for APIs and

- **The Reproducibility Standard:** While graphical tools like the RStudio Wizard are excellent for initial exploration, final decision-making workflows should always use **Programmatic Scripts** to ensure every step is auditable and repeatable Bivand et al. (2008), Huber (2025).
- **Memory Management:** For datasets larger than 2GB, avoid standard flat-file imports and prioritize **Relational Databases (DBI)** or **Partial Reading (rgdal / GDAL.open)** to prevent system crashes R Core Team (2015), Wickham and Grolemund (2016).
- **Structure Alignment:** Always select the package that best respects your data's inherent structure. For example, use **haven** for surveys to avoid losing categorical labels, or **lightr** for scientific spectrometry to preserve instrument metadata Luraschi (2023), Gruson et al. (2019).

3.4 Conclusion

Mastering data import is about matching the **right package** to the **right data structure** while respecting hardware limits. Choosing programmatic scripts ensures that the decision-making process is transparent, scalable, and reproducible Bivand et al. (2008), Huber (2025).

3.5 Class assignment

Download the following dataset from <https://alisoileimani71.github.io/>.

Tasks:

1. Import the dataset using readr, fread, and base R.
2. Measure import time using system.time().
3. Compare column types produced by each method.
4. Write a short paragraph explaining which method is best and why.

Affidavit

I hereby affirm that this submitted paper was authored unaided and solely by me. Additionally, no other sources than those in the reference list were used. Parts of this paper, including tables and figures, that have been taken either verbatim or analogously from other works have in each case been properly cited with regard to their origin and authorship. This paper either in parts or in its entirety, be it in the same or similar form, has not been submitted to any other examination board and has not been published. I acknowledge that the university may use plagiarism detection software to check my thesis. I agree to cooperate with any investigation of suspected plagiarism and to provide any additional information or evidence requested by the university.

Checklist:

- ☐ The handout contains 3-5 pages of text.
- ☐ The submission contains the Quarto file of the handout.
- ☐ The submission contains the Quarto file of the presentation.
- ☐ The submission contains the HTML file of the handout.
- ☐ The submission contains the HTML file of the presentation.
- ☐ The submission contains the PDF file of the handout.
- ☐ The submission contains the PDF file of the presentation.
- ☐ The title page of the presentation and the handout contain personal details (name, email, matriculation number).
- ☐ The handout contains a bibliography, created using BibTeX with an APA citation style.
- ☐ Either the handout or the presentation contains R code that proves the expertise in coding.
- ☐ The filled out Affidavit.

- The link to the presentation and the handout published on GitHub.

Ali Soleimani, 10.05.2026, Cologne

4 References

- Bivand, R. S., Pebesma, E. J., & Gómez-Rubio, V. (2008). *Applied spatial data analysis with r*. Springer Science & Business Media. <https://doi.org/https://doi.org/10.1007/978-0-387-78171-6>
- Chapman University. (2026). *R programming: Importing data files into r*. <https://libguides.chapman.edu/R/import>
- Chapter 2 - data import and export in r and python. (2019). In *R and python for oceanographers: A practical guide with applications* (pp. 23–47). Elsevier. <https://doi.org/10.1016/B978-0-12-813491-7.00002-6>
- Gruson, H., White, T., & Maia, R. (2019). Lightr: Import spectral data and metadata in r. *Journal of Open Source Software*, 4(43), 1857. <https://doi.org/10.21105/joss.01857>
- Huber, S. (2025). *How to use r for data science*. <https://hubchev.github.io/ds/>
- Luraschi, J. (2023). *Importing data with the RStudio IDE*. Posit Support. <https://support.posit.co/hc/en-us/articles/218611977-Importing-Data-with-the-RStudio-IDE>
- Ooms, J. (2014). The jsonlite package: A practical and consistent mapping between JSON data and r objects. *arXiv Preprint arXiv:1403.2805*.
- R Core Team. (2015). *R data import/export*. <https://cran.r-project.org/doc/manuals/r-release/R-data.pdf>
- Visne, I., Yildiz, A., Dilaveroglu, E., Vierlinger, K., Nöhammer, C., Leisch, F., & Kriegner, A. (2012). speedR: An r package for interactive data import, filtering and ready-to-use code generation. *Journal of Statistical Software*, 51(2), 1–12. <https://doi.org/10.18637/jss.v051.i02>
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- Wickham, H., & Grolemund, G. (2016). *R for data science: Import, tidy, transform, visualize, and model data*. O'Reilly Media, Inc.